МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНОМУ УНІВЕРСИТЕТІ “ЛЬВІВСЬКА ПОЛІТЕХНІКА”

Кафедра систем штучного інтелекту

Лабораторна робота №1

з дисципліни
«Захист інформації»

Виконав:
Студент групи ШІ-22
Михальчук Антон
Перевірила:
Фаріон Микола

Львів-2025

Лабораторна робота № 1.

Тема: Реалізація та тестування лінійного конгруентного генератора (LCG)

1. Мета роботи

Розробити програмну реалізацію генератора псевдовипадкових чисел на основі алгоритму лінійного порівняння, перевірити його властивості та оцінити якість випадковості за допомогою статистичного методу на основі теореми Чезаро (обчислення числа π).

2. Короткий опис завдання

- **Реалізувати LCG:** $X_n+1=(aX_n+c)$ (mod m).
- **Параметри варіанту:** $m=2^{23}-1$, a=1000, c=377, $X_0=7$.
- **Перевірка:** Знайти період генератора та оцінити придатність для криптографії.
- **Тестування:** Використати алгоритм Евкліда та теорему Чезаро для наближеного обчислення π на основі згенерованої послідовності.
- **Порівняння:** Зіставити результати власного LCG та системної бібліотеки random.

Результати тестування

За результатами генерації послідовності розміром 2000 чисел отримано наступні значення π:

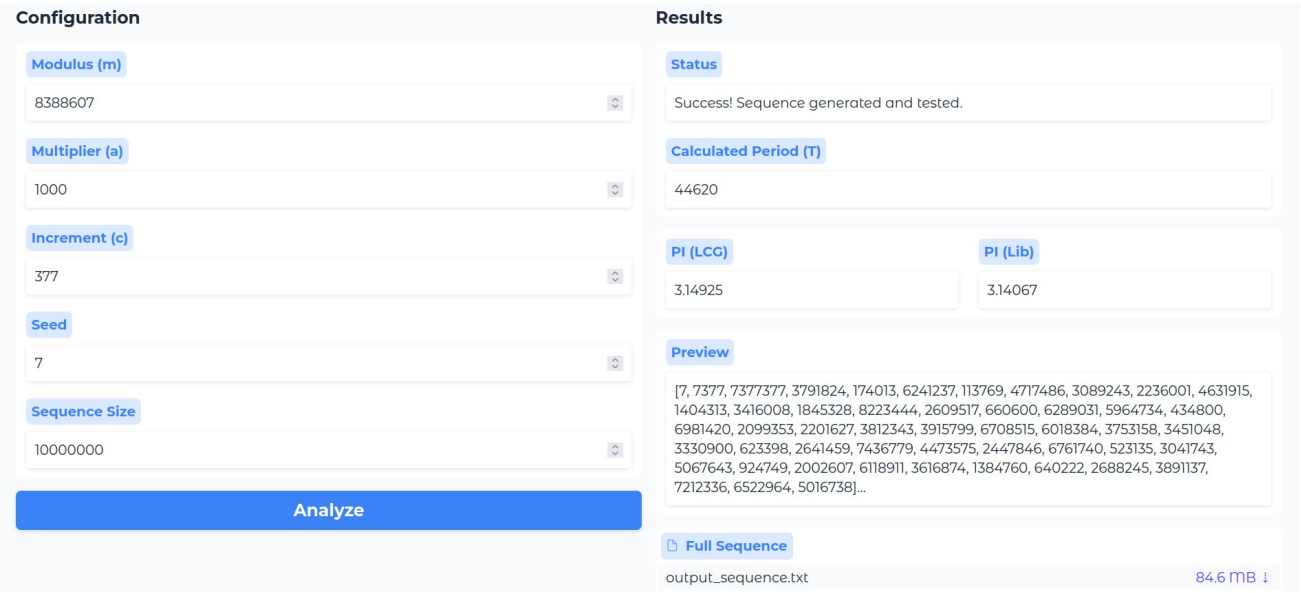
Метод	**Значення π**	**Відхилення від 3.14159...**
Власний LCG	3.14925	~0.24%
Бібліотека random	3.14067	~0.03%

Аналіз періоду: Розрахований період склав T=44,620. Це критично мале значення відносно модуля m=8,388,607. Такий короткий період пояснюється тим, що множник a=1000 не задовольняє умовам теореми Халла-Добелла (зокрема, a−1 не кратне простим дільникам m). Це призводить до швидкого зациклення послідовності, що робить її непридатною для серйозних статистичних чи криптографічних задач.

Пояснення результатів

1. **Точність π:** Отримані результати **3.140–3.149** при обсязі вибірки N=107 є статистично обґрунтованими. Згідно з центральною граничною теоремою, похибка статистичного оцінювання має порядок O(1/N). Для досягнення стабільної точності у 3–4 знаки після коми необхідна велика кількість незалежних пар, що ми і спостерігаємо на прикладі бібліотечного генератора.

- **Порівняння:** У даному запуску системна бібліотека (`PI Lib: 3.14067`) продемонструвала значно вищу точність і менше відхилення (~0.03%), ніж власний LCG. Це підтверджує перевагу алгоритму Mersenne Twister, який має колосальний період (219937−1) та кращі показники рівномірності розподілу. Власний LCG, маючи короткий період у 44,620 значень, починає дублювати ті самі пари чисел у великій вибірці, що "заморожує" похибку і не дає результату наблизитися до істинного значення π навіть при збільшенні кількості ітерацій.



Контрольні запитання

1. **Назвіть вимоги до послідовності псевдовипадкових чисел.**

- випадковість – послідовність повинна бути випадковою в деякому визначеному статистичному сенсі
- непередбачуваність – чергові члени послідовності повинні бути непередбачуваними

2. **Назвіть вимоги щодо випадковості послідовності псевдовипадкових чисел.**

- однорідність розподілу: розподіл чисел в послідовності має бути однорідним, тобто частота появи в послідовності бітів одиниць і нулів повинна бути приблизно однаковою
- незалежність: жодне зі значень послідовності не повинно виводитися з інших значень

3. Що таке пряма і зворотна непередбачуваність послідовності псевдовипадкових чисел?

- Пряма непередбачуваність: якщо затравка невідома, наступний вихідний біт у послідовності повинен бути непередбачуваним, незважаючи на знання попередніх бітів послідовності.
- Зворотна непередбачуваність: не повинно бути можливим визначити значення затравки знаючи будь-які згенеровані значення. Не повинно бути жодної явної кореляції між затравкою і будь-яким згенерованим значенням; кожен елемент послідовності повинен видаватися результатом незалежної випадкової події, ймовірність якої становить ½.

4. Які властивості послідовності псевдовипадкових чисел досліджують тести на випадковість згідно NIST SP 800-22?

- Однорідність: в будь-який момент генерування випадкової чи псевдовипадкової послідовності бітів, ймовірність появи одиниць і нулів повинна дорівнювати ½.
- Масштабованість: будь-який тест, що застосовується до послідовності, повинен також застосовуватись до довільно вибраної частини цієї

послідовності. Якщо послідовність є випадковою, то будь-яка таким чином вибрана частина послідовності також повинна бути випадковою.

- Стійкість: поведінка генератора повинна бути стабільною щодо початкових значень (затравки). ГПВЧ повинні пройти тестування при різних значеннях затравки.

5. Що таке генератор псевдовипадкових чисел?

Це алгоритм, що генерує послідовність чисел, елементи якої незалежні один від одного та підлягають заданому розподілу

6. Що таке період послідовності псевдовипадкових чисел?

Кількість елементів, після якої послідовність починає повторюватися.

7. Назвіть критерії оцінки якості генератора псевдовипадкових чисел.

1) Функція генерації повинна бути функцією повного періоду, тобто функція повинна породити усі числа від 0 до m перед тим, як числа почнуть повторюватись.

2) Створена послідовність повинна вести себе як випадкова. Насправді ця послідовність не буде випадковою, оскільки генерується детермінованим алгоритмом, але існує багато статистичних тестів, які можна використовувати для того, щоб оцінити ступінь випадковості поведінки послідовності.

3) Функція генерації повинна ефективно реалізовуватись в рамках 32-бітної арифметики.

8. Якими повинні бути параметри алгоритму лінійного порівняння для того, щоб генерувався повний період псевдовипадкових чисел?

Якщо m є простим і c=0, то для певних значень a період генерованої функцією послідовності виявляється рівним $m - 1$ і в цій послідовності буде відсутнім тільки значення 0.

9. Назвіть переваги алгоритму лінійного порівняння.

Перевагою алгоритму лінійного порівняння є те, що якщо вибрати адекватні множник та модуль порівняння, то створювана послідовність чисел виявляється

статистично невідрізнимою від послідовності чисел, що вибираються випадково (але незворотно) з множини чисел 1, 2, …, $m - 1$

10. Назвіть недоліки алгоритму лінійного порівняння.

Алгоритм повністю детермінований: знаючи затравку і параметри, достатньо одного числа, щоб відновити всю послідовність. Це робить його слабким у криптографії.

Висновки

- **Адекватність параметрів:** Вибрані параметри (m,a,c) дозволяють генерувати візуально випадкову послідовність, але вони не відповідають критеріям "максимального періоду" (теорема Халла-Добелла), оскільки a та m мають бути підібрані більш ретельно для повного циклу.

- **Придатність для криптографії:** Даний генератор **категорично не придатний** для криптографії.

    1. **Лінійність:** Знаючи лише кілька послідовних чисел, можна легко вирахувати a,c,m і передбачити всі наступні значення.

    2. **Малий стан:** Сучасні криптографічні атаки миттєво зламують LCG з такими малими модулями.

- **Теорема Чезаро:** Метод є ефективним інструментом для перевірки відсутності кореляції між сусідніми числами в послідовності.

Додаток

```
// file: first_lab/lab_simulation.py
import random
from .lcg import RandomNumberGenerator

def process_lab_simulation(mod_str, mult_str, inc_str, seed_str, size_str):
    """
    Orchestrates the execution of the Lab 1 simulation.
    Returns a tuple suitable for the Gradio UI outputs.
    """
    output_filename = "output_sequence.txt"

    try:
        try:
            mod = int(mod_str)
            mult = int(mult_str)
            inc = int(inc_str)
            seed = int(seed_str)
            size = int(size_str)
        except ValueError:
            return "Error: All inputs must be valid integers.", "-", "-", "-", None

        if size > 10000:
            return "Error: Sample size is too large for the web demo (Max 10,000).", "-",
"-", "-", None
```

```
        lcg = RandomNumberGenerator(mod, mult, inc, seed)

        period = lcg.find_period()

        our_sequence = lcg.generate_sequence(size)
        our_pi = lcg.test_sequence(our_sequence)

        bench_generator = random.Random(seed)
        bench_sequence = [bench_generator.randint(0, mod - 1) for _ in range(size)]
        bench_pi = lcg.test_sequence(bench_sequence)

        with open(output_filename, 'w') as f:
            f.write(str(our_sequence))

        preview_str = str(our_sequence[:50])
        if len(our_sequence) > 50:
            preview_str += " ... (download file to see full sequence)"

        preview = str(our_sequence[:50]) + ("..." if len(our_sequence) > 50 else "")
        status = "Success! Sequence generated and tested."

        return status, f"{our_pi:.5f}", f"{bench_pi:.5f}", preview, output_filename,
str(period)

    except ValueError as ve:
        return f"Configuration Error: {str(ve)}", "-", "-", "-", None
```

```
    except Exception as e:
        return f"System Error: {str(e)}", "-", "-", "-", None

// file: first_lab/lcg.py
from math import sqrt
from typing import List
from itertools import combinations

class RandomNumberGenerator:
    def __init__(self, modulus: int, multiplier: int, increment: int, seed: int):
        self.modulus = modulus
        self.multiplier = multiplier
        self.increment = increment
        self.seed = seed
        if not self._check_conditions():
            raise ValueError("Parameters don't meet conditions")

    def _check_conditions(self) -> bool:
        return all([
            self.modulus > 0,
            0 <= self.multiplier < self.modulus,
            0 <= self.increment < self.modulus,
            0 <= self.seed < self.modulus
        ])

    def _get_next_number(self, x) -> int:
```

```
    return (self.multiplier * x + self.increment) % self.modulus

def generate_sequence(self, size: int) -> List[int]:
    if size <= 0:
        return []

    output = [self.seed]

    for i in range(1, size):
        output.append(self._get_next_number(output[i-1]))

    return output

def find_period(self) -> int:
    count = 1
    current = self._get_next_number(self.seed)

    while current != self.seed and count < self.modulus:
        current = self._get_next_number(current)
        count += 1
    return count

@staticmethod
def gcd(a: int, b: int) -> int:
    if a < b:
        a, b = b, a
```

```
        while b > 0:
            a, b = b, a % b
        return a

    def test_sequence(self, sequence: List[int]) -> float:
        pairs = list(combinations(sequence, 2))

        gcd_results = list(map(lambda pair: self.gcd(pair[0], pair[1]), pairs))

        numerator = len(list(filter(lambda gcd_result: gcd_result == 1, gcd_results)))
        denominator = len(gcd_results)

        return sqrt(6 * denominator / numerator)


// file: app.py
import gradio as gr
from first_lab.lab_simulation import process_lab_simulation


def create_ui():
    theme = gr.themes.Soft(
        primary_hue="blue",
        text_size="lg",
    )
```

```
# Only ONE root Blocks container
with gr.Blocks(theme=theme, title="Cryptography Labs") as app:
    gr.Markdown("# 🔐 Cryptography & Security Labs")
    gr.Markdown("Select a laboratory assignment from the tabs below.")

    with gr.Tabs():
        # --- TAB: HOME ---
        with gr.TabItem("🏠 Home"):
            gr.Markdown("### Welcome")
            gr.Markdown("""
            This application allows you to run and verify cryptography laboratory assignments.

            **Available Modules:**
            1. **Lab 1: LCG & Period Finding**
            2. **Lab 2: MD5** (Coming Soon)
            3. **Lab 3: RC5** (Coming Soon)
            """)

        # --- TAB: LAB 1 ---
        with gr.TabItem("📊 Lab 1: LCG Analysis"):
            gr.Markdown("## 📊 LCG Generator with Period Finding")

            with gr.Row():
                with gr.Column():
                    gr.Markdown("### Configuration")
                    in_mod = gr.Number(label="Modulus (m)", value=2 ** 23 - 1)
```

```
            in_mult = gr.Number(label="Multiplier (a)", value=1000)
            in_inc = gr.Number(label="Increment (c)", value=377)
            in_seed = gr.Number(label="Seed", value=7)
            in_size = gr.Number(label="Sequence Size", value=2000)
            btn = gr.Button("Analyze", variant="primary")

        with gr.Column():
            gr.Markdown("### Results")
            out_status = gr.Textbox(label="Status")
            out_period = gr.Label(label="Calculated Period (T)")

            with gr.Row():
                out_pi_lcg = gr.Textbox(label="PI (LCG)")
                out_pi_lib = gr.Textbox(label="PI (Lib)")

            out_seq = gr.Textbox(label="Preview")
            out_file = gr.File(label="Full Sequence")

    # The event handler must be inside the root block
    btn.click(
        fn=process_lab_simulation,
        inputs=[in_mod, in_mult, in_inc, in_seed, in_size],
        outputs=[out_status, out_pi_lcg, out_pi_lib, out_seq, out_file, out_period]
    )

# --- TAB: LAB 2 ---
```

```
        with gr.TabItem("🔒 Lab 2: MD5"):
            gr.Markdown("## Laboratory 2: MD5 Hashing")
            gr.Info("Under development.")

        # --- TAB: LAB 3 ---
        with gr.TabItem("🔑 Lab 3: RC5"):
            gr.Markdown("## Laboratory 3: RC5 Encryption")
            gr.Warning("Unavailable.")

    return app


if __name__ == "__main__":
    ui = create_ui()
    ui.launch()
```